HTTPS on the Colibri device: findings

Oct 25, 2026 · @Niclas

Purpose and context

Hakkas on a Colibri device should serve its UI over https only. These findings say that this looks feasible on the hardware, and set out the certificate options and their trade-offs. Nothing here is decided; it is input for a discussion with an expert.

Hakkas is a flow-based-programming runtime written in C. Users edit live dataflow graphs from a browser UI, which hakkas serves from the device itself:

- **Two listeners today:** a web server on one port for the UI's files, and a WebSocket on the next port up for the live protocol (graph edits, values, logins).
- **Login:** a challenge-response scheme designed so the password never crosses the network, even without TLS.
- **TLS today:** specified as optional and off by default, for the MCU's RAM and the browser warnings a self-signed certificate causes. It is not implemented yet: with TLS switched on, hakkas refuses to start.
- **Why https now:** the device is to be reached only over https. The UI will also show pictures of the hardware loaded from the vendor's https site.

The figures below are estimates from documentation and experience, not measurements on the hardware.

The hardware

The W5500 does not stand in the way: Zephyr drives it as a plain network adapter, so TLS runs on the MCU like with any other adapter.

The Colibri 8 carrier board takes one MCU module and a communication module. Its Ethernet module uses the WIZnet W5500 over SPI, and the carrier has 16 MB of flash.

MCU module	Chip	Core	SRAM
MCU2	STM32H562	Cortex-M33, 250 MHz	640 KB

MCU module	Chip	Core	SRAM
MCU6	STM32H723 (the catalogue lists STM32H735RG; to be confirmed)	Cortex-M7, 550 MHz	564 KB
MCU7	RP2350	2 × Cortex-M33, 150 MHz	520 KB

- **The W5500 in Zephyr:** the chip has its own TCP/IP stack, but Zephyr's driver opens its socket 0 in **MACRAW** mode and passes raw Ethernet frames. The IP stack, TCP and TLS therefore all run on the MCU.
- **No TLS on the chip:** the W5500 has no TLS of its own, so offloading it is not an option either way.
- **Throughput:** the SPI link limits it to a few Mbit/s, ample for the UI and its live protocol.
- **Crypto hardware:** some of these chips have crypto accelerators (the STM32H73x has CRYP and HASH, the RP2350 SHA-256 and a TRNG); which ones mbedTLS can use on each is an open question.

The software

Zephyr has everything needed: TLS through its mbedTLS integration, offered as TLS sockets, and an HTTP server that does HTTPS and WebSockets.

- **TLS sockets:** an application opens a socket with a TLS protocol and attaches credentials (certificate and key) by a security tag. The credentials are buffers registered at runtime, so they can be loaded from flash or a configuration file rather than compiled in.
- **Zephyr's HTTP server:** supports HTTPS services and WebSocket resources. Hakkas has its own HTTP and WebSocket servers, written against its platform layer; they would use the same TLS sockets underneath. Whether to keep them or move to Zephyr's server is open.
- **Hakkas' rule:** no credentials are compiled into any build, and secrets are never logged. A certificate's private key counts as a secret.
- **Desktop and Raspberry Pi builds:** POSIX, with OpenSSL or mbedTLS available; not the subject of this document, but the same configuration should serve both.

Resource estimates

RAM is the real cost: roughly 100 KB with a few TLS connections at once, which fits all three MCUs. None of this is measured yet.

Item	Estimate	Why
Incoming record buffer, per connection	~16 KB	A TLS record can be 16 KB, and browsers don't support negotiating a smaller maximum (the max-fragment-length extension), so the server must accept full records
Outgoing buffer, per connection	a few KB	The server chooses its own record size
Handshake	a temporary peak of some KB per connection being opened	Key exchange and certificate signing
Total, connections capped at a few	~100 KB	Against 520–640 KB SRAM
Handshake time, ECDSA P-256 certificate	some tens of ms	RSA-2048 signing would take much longer on a Cortex-M33
Flash for mbedTLS	perhaps 100–200 KB	Depends on the cipher suites built in

- **Connections:** browsers open several connections in parallel to load a page's files. Capping the number of TLS connections makes the RAM predictable; how browsers behave when the cap is reached needs checking.
- **Session resumption** (tickets) would spare repeated full handshakes.

Certificate options

Three ways to give each device a certificate; they differ in browser warnings, offline use and what has to run around the device. They are not exclusive.

Option	Browser warning	Works offline	What it takes
Self-signed, made by the device on first boot	Yes, per device and per browser	Yes	Nothing
Own CA (the vendor's or the plant's)	None, once its root is installed on each PC	Yes	Signing each device's certificate at production or commissioning; installing one root certificate per PC
Let's Encrypt, through a vendor service	None, nothing to install	No: the device needs internet to renew	A public name per device and a service run by the vendor

Self-signed

- Works everywhere at once, with no infrastructure.
- The user must click through a warning for every device, in every browser, and again whenever the certificate changes.
- A page reached through a certificate warning gets less from some browsers (for example, Chrome refuses service workers on it).
- Firefox stores the exception per host and port, which matters for hakkas' second port (next section).

Own CA

- A CA root certificate is installed once on each PC that uses the devices; every device certificate it signs is then trusted.
- Works fully offline, and certificate lifetimes can be long.
- Common in industrial settings. The CA's key has to be kept safe, and devices need a way to receive their signed certificate.
- Browsers trust a device certificate only for the names and addresses it lists, so they have to be known when signing (an IP address can be listed).

Let's Encrypt

- **HTTP-01 is impossible** on a LAN: Let's Encrypt must fetch a file from the device on port 80 over the internet.
- **DNS-01 works:** each device gets a public name such as `<serial>.devices.<vendor domain>`, and public DNS points it at the device's LAN address (e.g. 192.168.1.40).
- **A vendor service does the ACME work:** the device makes its own key and sends only a certificate request. The service proves the name through DNS, gets the certificate and returns it. No DNS credentials sit on the device.
- **Renewal needs internet regularly.** The default lifetime is 90 days today, 64 days from 10 February 2027 and 45 days from 16 February 2028. The industry cap is 47 days from 15 March 2029.
- **Some routers block it:** DNS-rebinding protection (common in home routers and dnsmasq setups) refuses public names that resolve to private addresses.
- **Rate limits:** 50 certificates per registered domain per 7 days; a fleet needs an override, which takes weeks to arrange.
- **The names are public:** every certificate appears in Certificate Transparency logs, so device names, and any serial numbers in them, become visible.
- **Users must use the name,** not the IP address, to get a valid certificate.

Common to all three

The device needs a way to install a certificate and key, which hakkas would load from its configuration directory, and an ECDSA P-256 key is the cheap choice on these MCUs.

Consequences for hakkas

Hakkas' specification would change from TLS optional and off by default to https only on the device, and the WebSocket's separate port becomes a problem with self-signed certificates.

- **The separate WebSocket port:** hakkas serves the page on one port and the WebSocket on the next one up. Firefox stores a certificate exception per host *and port*, so clicking through the warning for the page doesn't cover the WebSocket port. A refused WebSocket fails silently, with no warning page to click through. Serving the WebSocket on the page's own port (e.g. at `/ws`) avoids this; with a CA or Let's Encrypt the problem doesn't arise.

- **Origin check:** hakkas compares a WebSocket's `Origin` with its own origin, scheme included. Its rules already foresee `https`; with one port the comparison gets simpler.
- **The login stays:** the challenge-response login was designed to be safe without TLS; with TLS it is a second layer.
- **Desktop development:** browsers treat `localhost` as a secure origin, so the desktop build can keep plain http there.
- **Pictures from the vendor:** the UI will show hardware pictures from an https URL. An https page can load them freely; plain http pictures would be blocked or upgraded.
- **Browser features:** IndexedDB, planned for caching those pictures, works with or without https. Service workers and the Cache API need a secure origin, which a page behind a certificate warning may not count as.

Open questions for the expert

1. Are the RAM estimates realistic for mbedTLS as a server to current browsers, and what is a sensible cap on concurrent TLS connections?
2. TLS 1.3 only, or 1.2 as well? Which cipher suites keep flash and RAM small while every current browser accepts them?
3. Which crypto accelerators on the STM32H562, STM32H723/H735 and RP2350 can mbedTLS use under Zephyr, and is it worth it?
4. Where should the private key live: in internal flash, on the carrier's 16 MB flash, or in a secure element? How is it protected?
5. Which certificate model fits industrial plants: self-signed, an own CA, Let's Encrypt, or a combination with one as default?
6. With an own CA: who runs it (the vendor or each plant), how are device certificates issued, and which names or IP addresses go in them?
7. With Let's Encrypt: is a vendor DNS-01 service worth it, given rebinding protection, rate limits and public names? Is there a better known pattern?
8. How should certificate expiry and renewal be handled on a device that is offline for long periods?
9. Should hakkas keep its own HTTP and WebSocket servers on TLS sockets, or move to Zephyr's HTTP server?
10. Is serving the WebSocket on the page's port the right fix for the per-port certificate exception, or is there another?

Sources

- Zephyr's W5500 driver, eth_w5500.c: socket 0 opened in MACRAW mode
- Zephyr HTTP server: HTTPS services and WebSocket resources
- Let's Encrypt: certificate lifetimes
- Linuxiac: Let's Encrypt to cut certificate lifetimes to 45 days by 2028: the 64- and 45-day dates
- Let's Encrypt: rate limits
- Let's Encrypt: challenge types: HTTP-01 and DNS-01

The chip figures, RAM estimates and browser behaviour are from general knowledge, not from these pages.